

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №1

«Классы и объекты. Использование конструкторов.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Определить пользовательский класс.
2. Определить в классе следующие конструкторы: без параметров, с параметрами, копирования.
3. Определить в классе деструктор.
4. Определить в классе компоненты-функции для просмотра и установки полей данных (селекторы и модификаторы).
5. Написать демонстрационную программу, в которой продемонстрировать все три случая вызова конструктора-копирования, вызов конструктора с параметрами и конструктора без параметров.

Пользовательский класс ЗАРПЛАТА

ФИО – string

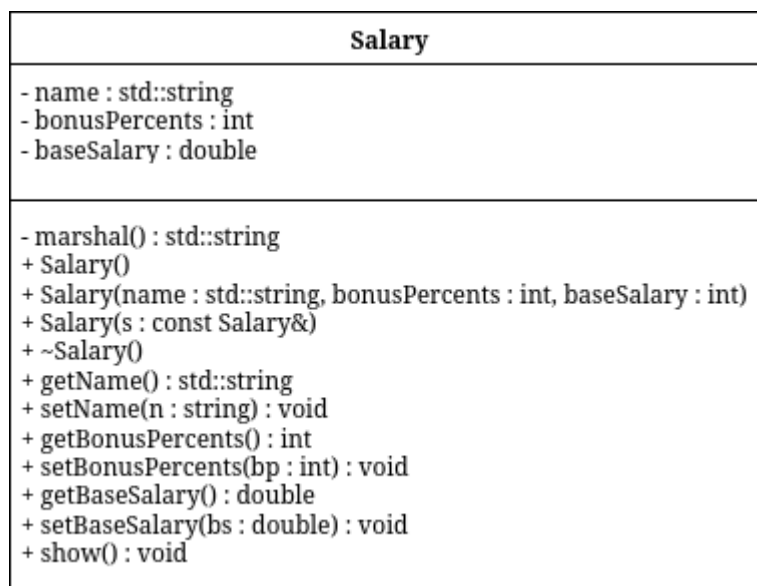
Оклад – double

Премия (% от оклада) – int

АНАЛИЗ.

1. Согласно принципам инкапсуляции поля класса должны быть приватными, а описанные в постановке методы — публичными.
2. Для работы с приватными полями реализуем гетеры и сетеры.
3. Для объявления переменных заданного класса реализуем конструкторы: обычный (принимает на вход значения полей класса), копирования и «пустой конструктор», выставляющий значения по умолчанию.

UML-ДИАГРАММА



КОД Salary.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_TOVAR_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_TOVAR_H
#include <string>

class Salary {
    std::string name;
    int bonusPercents;
    double baseSalary;
    std::string marshal();
public:
    Salary();
    Salary(std::string, int, double);
    Salary(const Salary&);
    ~Salary();
    std::string getName();
    void setName(std::string);
    int getBaseSalary();
    void setBaseSalary(double);
    double getBonusPercents();
    void setBonusPercents(int);
    void show();
};
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_TOVAR_H

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Salary.cpp

```
#include "Pair.h"
#include <iostream>

void Pair::init(double a, int b) {
    first = a;
    second = b;
}

void Pair::read() {
    std::cout << "First? ";
    std::cin >> first;
    std::cout << "Second? ";
    std::cin >> second;
}

void Pair::show() {
    std::cout << "First: " << first << "\nSecond: " << second << '\n';
}

double Pair::power() {
    double ans = first;
    for (int i = 1; i < second; i++) ans *= first;
    return ans;
}

double Pair::element(int a) {
    Pair tmp;
    tmp.init(second, a);
    return first * tmp.power();
}
```

КОД main.cpp

```
#include <iostream>;
#include "Salary.h"

using namespace std;

Salary makeSalary()
{
    string s;
    int i;
    double d;
```

```

    cout<<"Name? ";
    cin>>s;
    cout<<"Bonus percents? ";
    cin>>i;
    cout<<"Base salary? ";
    cin>>d;
    Salary t(s,i,d);
    return t;
}

int main() {
    Salary t1;
    Salary t2("Say Gabigi", 1, 15000);
    Salary t3=t2;
    t3.setName("Qebopi Coloyemuj");
    t3.setBonusPercents(2);
    t3.setBaseSalary(5000.0);
    t1=makeSalary();
    t1.show();
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Constructor without params:

Name:

Base salary: 0.000000

Bonus percents: 0

Final salary: 0.000000

Constructor with params:

Name: Say Gabigi

Base salary: 15000.000000

Bonus percents: 1

Final salary: 15150.000000

Copy constructor:

Name: Say Gabigi

Base salary: 15000.000000

Bonus percents: 1

Final salary: 15150.000000

Name? Fise_Zemex

Bonus percents? 25

Base salary? 100000

Constructor with params:

Name: Fise_Zemex

Base salary: 100000.000000

Bonus percents: 25

Final salary: 125000.000000

Destructor:

Name: Fise_Zemex

Base salary: 100000.000000

Bonus percents: 25

Final salary: 125000.000000

Name: Fise_Zemex

Base salary: 100000.000000

Bonus percents: 25

Final salary: 125000.000000

Destructor:

Name: Qebopi Coloyemuj

Base salary: 5000.000000

Bonus percents: 2

Final salary: 5100.000000

Destructor:

Name: Say Gabigi

Base salary: 15000.000000

Bonus percents: 1

Final salary: 15150.000000

Destructor:

Name: Fise_Zemex

Base salary: 100000.000000

Bonus percents: 25

Final salary: 125000.000000

ВОПРОСЫ

1. Для чего нужен конструктор?

Для инициализации полей класса.

2. Сколько типов конструкторов существует в C++?

Три: без параметров, с параметрами и копирования.

3. Для чего используется деструктор? В каких случаях деструктор описывается явно?

Для освобождения памяти, выделенной об объект класса. Используется явно при работе с указателями внутри класса.

4. Для чего используется конструктор без параметров? Конструктор с параметрами?

Конструктор копирования?

Конструктор без параметров используется для создания объекта класса со стандартными значениями полей. (пустого объекта)

Конструктор с параметрами используется для инициализация объекта с требуемыми параметрами.

Конструктор копирования используется для создания объекта класса с копированием полей другого объекта того же класса.

5. В каких случаях вызывается конструктор копирования?

1. При инициализации объекта другим объектом.

2. При передаче объекта в функцию по значению

3. При возврате объекта из функции

6. Перечислить свойства конструкторов.

- Не имеет тип

- Могут существовать несколько конструкторов с разными параметрами.

- Конструктор без полей имеет название "конструктор по умолчанию"

- Параметры конструктора с параметрами могут иметь любой тип, кроме типа класса.

- При явном отсутствии конструкторов, компилятор создает конструктор по умолчанию, однако он не сможет создать констатные или ссылочные поля (ошибка).

- Не наследуются

- Нельзя описать с модификаторами const, virtual, static.

- Конструкторы глобальных объектов вызываются до функции main();

7. Перечислить свойства деструкторов.

- Не имеет аргументов и возвращаемого значения

- Не наследуется

- Не может быть объявлен как const или static

- Может быть виртуальным

- Если деструктор явным образом не определен, компилятор автоматически создает пустой деструктор

8. К каким атрибутам имеют доступ методы класса?

Ко всем.

9. Что представляет собой указатель this?

Указатель на текущий объект класса.

10. Какая разница между методами определенными внутри класса и вне класса?

Ничем. Только кодстайл + удобство

11. Какое значение возвращает конструктор?

Никакое

12. Какие методы создаются по умолчанию?

Конструктор, деструктор

13. Какое значение возвращает деструктор?

Никакое

14. Дано описание класса

```
class Student
```

```
{
```

```
    string name;
```

```
    int group;
```

```
    public:
```

```
        student(string, int);
```

```
        student(const student&)
```

```
        ~student();
```

```
};
```

Какой метод отсутствует в описании класса?

Конструктор по умолчанию

15. Какой метод будет вызван при выполнении следующих операторов:

```
student*s;
```

```
s=new student;
```

Конструктор копирования

16. Какой метод будет вызван при выполнении следующих операторов:

```
student s("Ivanov",20);
```

Конструктор с параметрами

17. Какие методы будут вызваны при выполнении следующих операторов:

```
student s1("Ivanov",20);
```

```
student s2=s1;
```

Конструктор с параметрами, конструктор копирования

18. Какие методы будут вызваны при выполнении следующих операторов:

```
student s1("Ivanov",20);
```

```
student s2;
```

```
s2=s1;
```

Конструктор с параметрами, по умолчанию, конструктор копирования

19. Какой конструктор будет использоваться при передаче параметра в функцию print():

```
void print(student a)
```

```
{a.show();}
```

Конструктор копирования

20. Класс описан следующим образом:

```
class Student
```

```
{
```

```
    string name;
```

```
    int age;
```

```
    public:
```

```
    void set_name(string);
```

```
    void set_age(int );
```

```
    ....
```

```
};
```

```
Student p;
```

Каким образом можно присвоить новое значение атрибуту name объекта p?

```
set_name("name");
```